Arquitecturas y Organización de Computadoras I
Trabajo Práctico 7
Programación en lenguaje ensamblador MIPS. Convención de llamadas a procedimientos

Objetivo: Comprender la estructura de un programa en lenguaje ensamblador MIPS junto con la convención de llamada a procedimientos y funciones.

Recursos y Bibliografía:

Arq. MIPS Vol II.
Programa mipsx.
Apunte MIPS.

Tarea preliminar: Leer del Apunte de MIPS la *Sección 6 Convención de llamadas a procedimientos.* Comprenda también, al final de esa sección, el ejemplo presentado, con respecto al uso de registros, gestión de la pila, e instrucciones jal y jr.

Ejercicio 1. a. ¿Qué fallos comete el siguiente código de función (o subrutina)[1] **[*]** en cuanto a la convención de uso de registros y llamada a procedimiento y funciones?

```
fun:
      addiu   $sp,$sp,-32
      sw    $ra,28($sp)
      sw    $fp,24($sp)
      move  $fp,$sp
      li    $s0,1
      li    $t1,2
      jal   fun
      addi  $k0, $v0, 3
      add   $v0, $t1, $k0
      add   $v0, $s1, $k0
      move  $sp,$fp
      lw    $ra,28($sp)
      lw    $fp,24($sp)
      addiu   $sp,$sp,32
      jr    $ra
```

b. ¿Qué problemas pueden suceder si un compilador mal desarrollado (o un programador) no utiliza la convención de llamadas a procedimientos?. Mencione tres problemas que pueden surgir.

Ejercicio 2. Traduzca el programa en C, que se encuentra debajo, a un programa en lenguaje ensamblador MIPS, como lo haría un compilador. Respete la convención de llamada a procedimientos. Considere que char ocupa un byte e int una palabra. Verifique su correcto funcionamiento en *mipsx*.

a. ¿por qué las variables **x** y **n** de pow no son las mismas que las variables globales **x** y **n** definidas antes de **main**?
b. Indique cuales son las direcciones efectivas de **x** y **n** global, y de las variables **i** y **pow** en **pow()**.
c. ¿Cuál es la dirección del último byte del segmento de texto? ¿Cuántas pseudo instrucciones tiene el programa?
d. Indique las direcciones de inicio de los segmentos de pila, segmento de texto y segmento de datos del programa.

[1] [*] Subrutina o función: https://en.wikipedia.org/wiki/Function_(computer_programming)

```
int pow(char x, char n)
{
        int i;
        int pow = 1;

        for (i=0; i<n; i++)
                pow = pow * x;

        return pow;
}


char x = 30;
char n = 55;
int resultado = 0;

void main(void)
{
        resultado = pow(x, n);
}
```

Ejercicio 3. Desarrollar una función llamada strlen que reciba como argumento la dirección de una cadena de texto (terminada con el caracter nulo) y devuelva la longitud de la cadena. Respete la convención de llamada a procedimiento. Verifique el funcionamiento de su subrutina desarrollada llamándola desde main en mipsx.

Ejercicio 4. Desarrollar una función que calcule el factorial de un número, utilizando la definición recursiva, y respetando la convención de llamada a procedimiento.

```
factorial(0) = 1
factorial(N) = N * factorial(N-1)
```